



Time Series Analysis and Forecasting

Seminar 11: LLMs and Foundation Models for Time Series



Daniel Traian PELE

Bucharest University of Economic Studies

IDA Institute Digital Assets

Blockchain Research Center

AI4EFin Artificial Intelligence for Energy Finance

Romanian Academy, Institute for Economic Forecasting

MSCA Digital Finance

Seminar Outline

- ▣ **Part I: Year 3 Level** – Self-attention, positional encoding, architectures, tokenization
- ▣ **Part II: Master Level** – Zero-shot decisions, LoRA fine-tuning, benchmark interpretation
- ▣ **Part III: PhD Extension** – Scaling laws, cross-domain transfer, ensemble strategies
- ▣ **Discussion Topics** – Foundation models vs. ARIMA, interpretability, data privacy
- ▣ **AI-Assisted Exercise** – Critical thinking with AI tools
- ▣ **Take-Home Exercises** – Extended practice
- ▣ **Summary** – Key concepts by level
- ▣ **Bibliography** – References and code

Exercise 1: Self-Attention Computation

Question

The scaled dot-product attention is defined as $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$. Given the 3×3 matrices ($d_k = 3$):

$$Q = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \quad K = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Exercise

- **(A)** Compute $QK^\top$ and interpret each entry as a “similarity score” between query i and key j .
- **(B)** Scale by $\sqrt{d_k}$, apply row-wise softmax, and interpret the resulting attention weights.
- **(C)** Multiply the weight matrix by V . What does each row of the output represent?

Answer on next slide...

Exercise 1: Solution

Solution

- (A) $QK^T = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 2 \end{pmatrix}$. Entry (i, j) measures how much query i “attends to” key j . E.g. row 1 assigns the highest raw score (2) to key 2.
- (B) Dividing by $\sqrt{3} \approx 1.732$: $S = \begin{pmatrix} 0.577 & 1.155 & 0.577 \\ 0.577 & 0 & 0.577 \\ 0.577 & 0.577 & 1.155 \end{pmatrix}$. Row-wise softmax yields weights W , e.g. row 1 $\approx (0.25, 0.44, 0.31)$ — position 1 attends most strongly to position 2. Scaling prevents the softmax from saturating to one-hot for large d_k .
- (C) Output = $W \cdot V$: each output row is a **weighted average** of the value vectors, where the weights encode the learned relevance of each position. This is how the Transformer aggregates contextual information without recurrence.

Exercise 2: Positional Encoding

Question

The sinusoidal positional encoding (Vaswani et al., 2017) is defined as:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}}), \quad PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Exercise

Compute the positional encodings for positions $pos = 0, 1, 2, 3$ with $d_{\text{model}} = 4$ (so $i \in \{0, 1\}$).

- **(A)** Fill in the 4×4 PE matrix. What patterns do you observe across rows and columns?
- **(B)** Why does each position receive a unique encoding, even though $\sin$ and $\cos$ are periodic?
- **(C)** Show that relative positions remain computable: PE_{pos+k} can be written as a linear transformation of PE_{pos} .

Answer on next slide...

Exercise 2: Solution

Solution

- (A) With $\omega_0 = 1$, $\omega_1 = 1/100$:

$$PE = \begin{pmatrix} \sin(0) & \cos(0) & \sin(0) & \cos(0) \\ \sin(1) & \cos(1) & \sin(0.01) & \cos(0.01) \\ \sin(2) & \cos(2) & \sin(0.02) & \cos(0.02) \\ \sin(3) & \cos(3) & \sin(0.03) & \cos(0.03) \end{pmatrix} \approx \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0.841 & 0.540 & 0.010 & 1.000 \\ 0.909 & -0.416 & 0.020 & 1.000 \\ 0.141 & -0.990 & 0.030 & 1.000 \end{pmatrix}$$

Low-index dimensions oscillate fast (high frequency); high-index dimensions change slowly.

- (B) Each position is encoded by sin / cos pairs at **geometrically spaced** wavelengths (2π to $2\pi \cdot 10000$). The combination of many different frequencies creates a unique “fingerprint” for each position, analogous to how a number has a unique representation in multiple bases.
- (C) Since $\sin(\omega(pos + k)) = \sin(\omega pos) \cos(\omega k) + \cos(\omega pos) \sin(\omega k)$, the encoding at $pos + k$ is a **rotation matrix** applied to the encoding at pos . This enables the model to learn relative positions via linear projections.

Exercise 3: Architectural Comparison

Question

Three foundation models for time series use different Transformer architectures:

Model	Architecture	Base
Chronos	Encoder-Decoder	T5
TimesFM	Decoder-only	Custom
Moirai	Encoder-only	BERT-style

Exercise

- (A) Why is a **decoder-only** architecture natural for autoregressive forecasting?
- (B) Why does an **encoder-decoder** allow flexible input/output lengths?
- (C) How can an **encoder-only** model (Moirai) perform forecasting if it has no autoregressive decoder?

Answer on next slide...

Exercise 3: Solution

Solution

- **(A)** Decoder-only models use **causal (left-to-right) attention**: token t can only attend to tokens $\leq t$. This mirrors the autoregressive factorization $p(y_{T+1}, \dots, y_{T+H}) = \prod_h p(y_{T+h} \mid y_{\leq T+h-1})$. TimesFM generates one forecast step at a time, conditioning on all prior values.
- **(B)** The encoder processes the full context window with **bidirectional attention** (each token sees all others), then the decoder attends to the encoder output via **cross-attention** while generating the forecast autoregressively. Input and output can have different lengths because they are processed by separate modules. Chronos encodes 512 context tokens and decodes up to 64 forecast tokens.
- **(C)** Moirai uses **masked prediction**: future positions are replaced with [MASK] tokens, and the encoder predicts their values using bidirectional context from the observed positions. This is analogous to BERT's masked language modeling — the model fills in “blanks” rather than generating left-to-right.

Exercise 4: Tokenization Strategies

Question

Two tokenization strategies for time series: **Patching** (PatchTST): group P consecutive time steps into one token. **Quantization** (Chronos): discretize values into B bins and use a vocabulary of B tokens.

Exercise

Consider a series of $T = 512$ time steps.

- (A) With patch size $P = 32$, compute the number of tokens and the attention complexity reduction compared to point-level tokenization ($P = 1$). (Hint: self-attention is $O(n^2)$.)
- (B) With Chronos-style quantization using $B = 4096$ bins uniformly over $[\min, \max]$, compute the maximum quantization error. What is the resolution for a series with range $[0, 100]$?
- (C) Which strategy is better for high-frequency financial data (tick-level prices) and why?

Answer on next slide...

Exercise 4: Solution

Solution

- **(A)** Number of tokens: $512/32 = 16$. Self-attention complexity: $O(16^2) = O(256)$ vs. $O(512^2) = O(262,144)$ for $P = 1$. Reduction factor: $P^2 = 32^2 = 1024\times$. This makes long-context forecasting computationally feasible.
- **(B)** Bin width: $100/4096 \approx 0.0244$. Maximum quantization error: half a bin width = 0.0122. This corresponds to 0.012% of the range — negligible for most applications. However, Chronos actually uses **mean scaling** before quantization, so the effective resolution depends on the scale of each series.
- **(C) Quantization is better** for high-frequency financial data: (i) Patching with $P = 32$ would merge 32 ticks into one token, losing critical microstructure information (bid-ask bounces, order flow patterns). (ii) Quantization preserves the **temporal ordering** of each tick while enabling the model to use a discrete vocabulary. (iii) For financial data, a smaller patch size ($P = 4$ or $P = 8$) could be used, but this increases attention cost.

Exercise 5: Zero-Shot Decision Framework

Question

A retail company has 50,000 product-level time series, each with ~ 200 weekly observations at mixed frequencies (weekly sales, monthly promotions, quarterly targets). They want to forecast 12 weeks ahead.

Exercise

- **(A)** Apply the decision tree: (1) Is there enough domain-specific data to fine-tune? (2) Are the series sufficiently different from pre-training data to require adaptation? Recommend **zero-shot**, **fine-tuned**, or **traditional** models.
- **(B)** Estimate the computational cost: fine-tuning a 200M-parameter model on 50K series (10 epochs, batch size 32) vs. zero-shot inference. Assume 1 GPU-hour per 100K gradient steps.
- **(C)** Recommend a specific model (Chronos, TimesFM, or Moirai) and justify your choice based on context length, probabilistic output, and pre-training data composition.

Answer on next slide...

Exercise 5: Solution

Solution

- ▣ **(A)** 200 observations per series is relatively short, but $50\text{K series} \times 200 = 10\text{M}$ data points in total — enough to fine-tune. However, retail data is well-represented in pre-training corpora (Monash, M5). **Recommendation:** start with **zero-shot**, then fine-tune only if performance is unsatisfactory. Use ARIMA/ETS as baselines.
- ▣ **(B)** Fine-tuning: $50,000 \times 200/32 \approx 312,500$ steps per epoch $\times 10$ epochs = 3.125M steps $\approx$ **31 GPU-hours**. Zero-shot inference: 50,000 series, ~ 0.1 seconds each $\approx$ **1.4 hours on 1 GPU**. Zero-shot is $\sim 22\times$ cheaper — a compelling advantage when it works well enough.
- ▣ **(C) Chronos-Large (710M):** trained on 27 diverse datasets including retail; supports context up to 512 tokens; provides full **probabilistic forecasts** (quantiles) via the quantization approach; T5 encoder-decoder handles variable horizons. TimesFM is also suitable but provides only point forecasts. Moirai handles mixed frequencies natively via its any-variate framework.

 TSA_ch11_zero_shot

Exercise 6: LoRA Fine-Tuning

Question

A foundation model has 200M parameters. You want to adapt it to energy forecasting data using LoRA (Low-Rank Adaptation) with rank $r = 8$ applied to all attention weight matrices. Each attention layer has $W_Q, W_K, W_V, W_O \in \mathbb{R}^{512 \times 512}$, and the model has 12 layers.

Exercise

- (A) Compute the number of trainable LoRA parameters. Each LoRA adapter decomposes $\Delta W = BA$ where $B \in \mathbb{R}^{512 \times 8}$, $A \in \mathbb{R}^{8 \times 512}$.
- (B) What fraction of the original parameters are trained? Compare memory requirements.
- (C) Explain how LoRA mitigates **catastrophic forgetting** compared to full fine-tuning.

Answer on next slide...

Exercise 6: Solution

Solution

- ▣ **(A)** Per adapter: $512 \times 8 + 8 \times 512 = 8,192$ parameters. Per layer: $4 \times 8,192 = 32,768$ (four attention matrices). Total: $12 \times 32,768 = 393,216$ trainable parameters.
- ▣ **(B)** Fraction: $393,216 / 200,000,000 \approx 0.2\%$. Full fine-tuning requires storing gradients and optimizer states for all 200M parameters (~ 2.4 GB in FP32). LoRA requires optimizer states only for 393K parameters (~ 4.7 MB), plus the frozen model weights loaded once. GPU memory drops from ~ 6 GB to ~ 1.5 GB.
- ▣ **(C)** In LoRA, the original weights W are **frozen**: $W' = W + \Delta W = W + BA$. The pre-trained knowledge is preserved exactly in W ; only the low-rank residual BA is learned. If the fine-tuning data is unrepresentative, one can simply discard BA and recover the original model. Full fine-tuning modifies all weights directly, risking **catastrophic forgetting** of generalizable temporal patterns.

Exercise 7: Benchmark Interpretation

Question

The following table shows weighted quantile loss (WQL, lower is better) on the Monash benchmark:

Model	Short-horizon	Long-horizon	Overall
ARIMA (auto)	0.112	0.198	0.155
Chronos-Large	0.098	0.142	0.120
TimesFM	0.101	0.139	0.120

Exercise

- **(A)** Compute the relative improvement of Chronos over ARIMA for short-horizon and long-horizon.
- **(B)** On which dataset types does ARIMA tend to outperform foundation models? Why?
- **(C)** Propose a model selection strategy that combines statistical and foundation models.

Answer on next slide...

Exercise 7: Solution

Solution

- ▣ **(A)** Short-horizon: $(0.112 - 0.098)/0.112 = 12.5\%$ improvement. Long-horizon: $(0.198 - 0.142)/0.198 = 28.3\%$ improvement. Foundation models excel at longer horizons where ARIMA's linear extrapolation deteriorates.
- ▣ **(B)** ARIMA wins on: **(i)** short, low-frequency series ($n < 100$) where foundation models lack sufficient context; **(ii)** strongly seasonal series with simple structure (airline-type) where the correct ARIMA order captures the DGP exactly; **(iii)** series from domains poorly represented in pre-training data. ARIMA's strength is parsimony when the true model is indeed linear.
- ▣ **(C) Cascade strategy:** (1) Run zero-shot foundation model + auto-ARIMA on a holdout set. (2) For each series, select the model with lower holdout loss. (3) If the foundation model wins on $> 70\%$ of series, use it globally (simpler deployment). (4) Otherwise, maintain per-series selection — this typically beats both individual models by 5–10%.

Exercise 8: Scaling Laws

Question

Chronos comes in four sizes: Mini (20M), Small (46M), Base (200M), and Large (710M). On the Monash benchmark, the WQL scores are 0.128, 0.125, 0.121, and 0.120 respectively — a nearly flat curve beyond 200M parameters.

Exercise

- ▣ **(A)** In NLP, scaling laws show $L \propto N^{-\alpha}$ (loss decreases as a power law in model size N). Contrast this with the Chronos results. What is different?
- ▣ **(B)** Explain why time series scaling saturates: consider data diversity, signal-to-noise ratio, and the inherent complexity of temporal patterns vs. language.
- ▣ **(C)** Propose what could break the saturation and re-enable scaling.

Answer on next slide...

Exercise 8: Solution

Solution

- ▣ **(A)** In NLP, GPT-3 (175B) consistently outperforms GPT-2 (1.5B) because language has vast combinatorial complexity. For Chronos, going from 200M to 710M ($3.5\times$) improves WQL by only 0.8% ($0.121 \rightarrow 0.120$). The scaling exponent $\alpha \approx 0$ for large models — fundamentally different from the NLP power law.
- ▣ **(B)** Three factors: **(i) Limited data diversity**: pre-training used ~ 27 datasets ($\sim 100K$ series); NLP uses trillions of tokens. Model capacity exceeds data complexity. **(ii) Low signal-to-noise**: time series have high irreducible noise, creating a performance floor that more parameters cannot breach. **(iii) Simpler patterns**: temporal patterns (trends, seasonality, AR dynamics) require far fewer parameters than natural language semantics.
- ▣ **(C)** Breakthrough paths: **(i)** massive increase in training data — billions of series from IoT, financial markets, weather; **(ii) multimodal pre-training** combining time series with text metadata; **(iii)** synthetic data augmentation using physics-informed simulations.

Exercise 9: Cross-Domain Transfer

Question

A model pre-trained on energy consumption data (hourly electricity load across 370 substations, 4 years) is applied zero-shot to forecast weekly retail sales for 3,000 stores.

Exercise

- (A) What temporal patterns **transfer** from energy to retail? Why would a model learn these as domain-agnostic features?
- (B) What aspects of the retail domain **do not transfer** from energy pre-training?
- (C) Design a controlled experiment to measure the effectiveness of cross-domain transfer.

Answer on next slide...

Exercise 9: Solution

Solution

- **(A) Transferable patterns:** (i) *Weekly seasonality*: both energy and retail show 7-day cycles. (ii) *Holiday effects*: demand spikes/dips around holidays. (iii) *Trend and level-shift detection*: the model learns to recognize structural breaks. (iv) *Autoregressive dynamics*: short-range temporal dependencies are universal. These are domain-agnostic because the Transformer learns abstract temporal features, not domain-specific causal mechanisms.
- **(B) Non-transferable:** (i) *Promotion effects*: retail sales spike during discounts — energy has no analog. (ii) *Intermittent demand*: many retail products have zero-inflated sales; energy load is always positive. (iii) *Cross-series dependencies*: store cannibalization and substitution effects are retail-specific. (iv) *Frequency mismatch*: hourly energy patterns do not directly map to weekly retail.
- **(C) Experiment:** (1) Train model A on energy only, B on retail only, C on both. (2) Evaluate all three on held-out retail test set. (3) Compute $\Delta = \text{WQL}(A) - \text{WQL}(B)$: if Δ is small, transfer is effective. (4) If C beats B, energy pre-training provides **positive transfer**. (5) Control for model size and compute budget.

Exercise 10: Ensemble Strategy

Question

You have two forecasters: ARIMA providing a point forecast $\hat{y}_{T+h}^A$ with known RMSE, and Chronos providing a full predictive distribution $\hat{F}_{T+h}^C$ (quantiles at levels 0.1, 0.5, 0.9). You want to combine them.

Exercise

- (A) Propose a weighting scheme for the point forecasts: $\hat{y}_{T+h}^E = w \hat{y}_{T+h}^A + (1 - w) \hat{y}_{T+h}^C$. How do you estimate w ?
- (B) How can you use **conformal prediction** to calibrate the combined forecast's prediction intervals?
- (C) Under what conditions does the ensemble beat both individual components?

Answer on next slide...

Exercise 10: Solution

Solution

- **(A) Inverse-variance weighting:** $w = \sigma_C^2 / (\sigma_A^2 + \sigma_C^2)$, where σ_A^2, σ_C^2 are estimated on a validation set. If errors are uncorrelated, the ensemble RMSE is $\sigma_E = \sigma_A \sigma_C / \sqrt{\sigma_A^2 + \sigma_C^2} < \min(\sigma_A, \sigma_C)$. In practice, use **time-series cross-validation** (expanding window) to estimate the variances and optimize w by minimizing holdout loss.
- **(B) Conformal prediction:** (1) On a calibration set of n past forecasts, compute residuals $e_i = |y_i - \hat{y}_i^E|$. (2) Sort residuals and take the $(1 - \alpha)(1 + 1/n)$ -th quantile $\hat{q}$. (3) The prediction interval $[\hat{y}_{T+h}^E - \hat{q}, \hat{y}_{T+h}^E + \hat{q}]$ has guaranteed $(1 - \alpha)$ marginal coverage. This is **distribution-free** — it does not assume Gaussian errors.
- **(C)** The ensemble beats both when: (i) forecast **errors are weakly correlated** (ARIMA captures linear patterns; Chronos captures nonlinear ones); (ii) neither model dominates across all horizons or series; (iii) the weighting is estimated on **out-of-sample** data (in-sample optimization overfits w). The ensemble fails when both models make the same systematic errors.

Discussion: Will Foundation Models Replace ARIMA?

Key Question

Foundation models match or beat ARIMA on large benchmarks. Does this mean ARIMA is obsolete?

Arguments For Replacement

- ▣ Zero-shot models require no model selection (no p, d, q tuning)
- ▣ Foundation models handle non-linear patterns, regime changes, and long-range dependencies
- ▣ One model serves thousands of series — massive operational simplification

Arguments Against

- ▣ ARIMA is **interpretable**: each coefficient has a clear statistical meaning
- ▣ ARIMA is **optimal** for linear Gaussian processes (Wold decomposition)
- ▣ ARIMA runs on a laptop in milliseconds; foundation models need GPUs
- ▣ ARIMA provides **analytical** confidence intervals; foundation model intervals require calibration
- ▣ **Complementary, not competitive**: the best strategy is often an ensemble

Discussion: Black Box vs. Interpretability

Key Question

Foundation models are “black boxes.” When is this acceptable, and when is it a dealbreaker?

When Black Boxes Are Acceptable

- ▣ **Demand forecasting:** accuracy is the primary objective; explanations are secondary
- ▣ **Anomaly detection:** the model flags outliers; humans investigate causes
- ▣ **High-frequency trading:** speed matters more than explainability
- ▣ **Research:** comparing forecasting performance across model classes

When Interpretability Is Required

- ▣ **Regulatory compliance:** Basel III/IV requires explainable risk models
- ▣ **Central banking:** monetary policy decisions must be justified to the public
- ▣ **Healthcare:** drug demand forecasting has patient safety implications
- ▣ **Legal liability:** “the AI said so” is not a defense in court

Discussion: Data Privacy in API-Based Models

Key Question

TimeGPT (Nixtla) operates as a cloud API: you send your time series data to their servers. What are the implications?

Privacy and Security Concerns

- ▣ **Data sovereignty:** financial data may be subject to GDPR, CCPA, or banking regulations
- ▣ **Intellectual property:** proprietary signals could be reverse-engineered from API logs
- ▣ **Vendor lock-in:** if the API goes offline, your forecasting pipeline breaks
- ▣ **Reproducibility:** model updates by the provider can change results silently

Mitigation Strategies

- ▣ Use **open-weight models** (Chronos, Moirai, Lag-Llama) — run locally, full control
- ▣ Apply **differential privacy:** add calibrated noise before sending data to the API
- ▣ Negotiate **data processing agreements** (DPAs) with API providers
- ▣ Maintain a **local fallback** model (ARIMA/ETS) for business continuity

AI Exercise: Chronos Forecasting with Code Verification

Prompt to test in ChatGPT / Claude / Copilot

"Write Python code to forecast the next 24 months of the AirPassengers dataset using the Chronos model from Amazon. Include: loading the pre-trained model, preparing the context window, generating probabilistic forecasts (10th, 50th, 90th percentiles), and plotting the results with prediction intervals."

Exercise — Verify the AI-generated code critically:

1. Does the code install the correct package (`chronos-forecasting`) and load a valid checkpoint?
2. Is the **context window** handled correctly? (Chronos supports up to 512 tokens — truncate/pad?)
3. Are forecast quantiles generated with correct `num_samples` and `prediction_length`?
4. Does evaluation use **out-of-sample** data, or does it accidentally evaluate on training data?
5. Are prediction intervals correctly computed from the sampled trajectories?

Warning: AI-generated code may run without errors and look professional. *That does not mean it is correct.*

 TSA_ch11_chronos_demo

Take-Home Exercises

1. **Theoretical:** Prove that multi-head attention with h heads and head dimension $d_k = d_{\text{model}}/h$ has the same total parameter count as single-head attention with dimension d_{model} , but can attend to information from different representation subspaces simultaneously.
2. **Computational:** Implement patching for a time series of $T = 1024$ observations:
 - ▶ Compute the number of tokens for patch sizes $P \in \{8, 16, 32, 64\}$
 - ▶ Measure wall-clock time for self-attention at each patch size
 - ▶ Plot the accuracy-efficiency tradeoff on a forecasting task of your choice
3. **Applied:** Compare Chronos (zero-shot) vs. auto-ARIMA vs. ETS on 10 series from the M4 monthly dataset:
 - ▶ Report MASE and sMAPE for each model and each series
 - ▶ On how many series does Chronos win? Characterize the “winning” series
4. **Critical Thinking:** Foundation models are trained on publicly available datasets. If a practitioner’s proprietary data has fundamentally different dynamics (e.g. cryptocurrency order flow), can zero-shot inference be trusted? What safeguards should be in place?

Exercise Solution Hints

Hints

1. Multi-head: h heads each with $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Total: $h \cdot 3 \cdot d_{\text{model}} \cdot d_k + d_{\text{model}}^2 = 4d_{\text{model}}^2$, same as single-head ($3d_{\text{model}}^2 + d_{\text{model}}^2$). The benefit is h independent attention patterns.
2. Tokens: $1024/P$. Attention complexity: $O((1024/P)^2 \cdot P)$ per layer. Use `torch.nn.MultiheadAttention` and `time.perf_counter()`. Expect diminishing returns beyond $P = 32$.
3. Use the `datasetsforecast` package for M4 data. Chronos tends to win on series with complex seasonality or level shifts; ARIMA wins on short, simple autoregressive series. Report paired Wilcoxon test for statistical significance.
4. Key safeguard: **always benchmark against a simple baseline on your own data**. If zero-shot performance is within 5% of ARIMA, the distribution shift is manageable. If the gap is $> 20\%$, the pre-training data is too different — fine-tuning or domain-specific models are needed.

Summary: Chapter 11

Part	Exercises	Key Concepts
Year 3	Ex. 1–4	Self-attention mechanics; positional encoding; encoder vs. decoder vs. enc-dec architectures; patching vs. quantization tokenization
Master	Ex. 5–7	Zero-shot vs. fine-tuning decision framework; LoRA parameter-efficient adaptation; benchmark interpretation and model selection
PhD	Ex. 8–10	Scaling law saturation in TS vs. NLP; cross-domain transfer patterns; ensemble with conformal calibration

Key insight: Foundation models are a powerful *complement* to statistical methods — use zero-shot for rapid prototyping, fine-tune for domain adaptation, and ensemble for production accuracy

Questions?

Bibliography I

Transformer Foundations

- ▣ Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- ▣ Hu, E.J., et al. (2022). LoRA: Low-rank adaptation of large language models. *ICLR 2022*.
- ▣ Nie, Y., et al. (2023). A time series is worth 64 words: Long-term forecasting with Transformers. *ICLR 2023*.

Foundation Models for Time Series

- ▣ Ansari, A.F., et al. (2024). Chronos: Learning the language of time series. *arXiv:2403.07815*.
- ▣ Das, A., et al. (2024). A decoder-only foundation model for time-series forecasting. *ICML 2024*.
- ▣ Garza, A., & Mergenthaler-Canseco, M. (2024). TimeGPT-1. *arXiv:2310.03589*.

Bibliography II

Additional Foundation Models

- ▣ Rasul, K., et al. (2024). Lag-Llama: Towards foundation models for probabilistic time series forecasting. *arXiv:2310.08278*.
- ▣ Woo, G., et al. (2024). Unified training of universal time series forecasting transformers. *ICML 2024*.

Online Resources and Code

- ▣ **Quantlet**: <https://quantlet.com> — Code repository for quantitative methods
- ▣ **Quantinar**: <https://quantinar.com> — Quantitative methods learning platform
- ▣ **GitHub TSA**: https://github.com/danpele/TSA/tree/main/TSA_ch11 — Python code for this seminar
- ▣ **Chronos**: <https://github.com/amazon-science/chronos-forecasting> — Amazon's Chronos models
- ▣ **Moirai**: <https://github.com/SalesforceAIResearch/uni2ts> — Salesforce's Moirai models

Thank You!

Questions?

Seminar materials are available at: <https://danpele.github.io/Time-Series-Analysis/>



Quantlet



Quantinar